

Logic-Based Benders Method for Multi-Skill Resource-Constrained Project Scheduling with Allocation and Skill Flexibility

Carla Juvin^a, Christian Artigues^b, Pierre Lopez^b, Rémi Brux^b

^a*TBS Business School, 1 Place Alphonse Jourdain, 31068 Toulouse, France*

^b*Univ Toulouse, CNRS, LAAS, 7 Avenue du Colonel Roche, 31031 Toulouse, France*

Abstract

In this paper, we study a multi-skill resource-constrained project scheduling problem with partial preemption, where we assume that a worker can cover several skills and the worker allocation can change during the processing of activities. This variant allows for the modelling of practical situations involving worker flexibility that were not previously considered in the literature. **By adapting a less common decomposition scheme, we propose a new exact logic-based Benders decomposition method that reverses the standard decomposition structure. The master problem is a scheduling problem, where the allocation of workers is relaxed into cumulative resource constraints.** This master problem is solved by Constraint Programming. The subproblems consist in verifying, using Integer Linear Programming, that worker allocation is possible at any instant. Logical feasibility cuts are issued from the subproblems in the form of new cumulative resource constraints. We extend the decomposition method with preprocessing and more general cuts. The beneficial effects of these improvements are demonstrated by computational experiments. Due to the high-quality lower bound obtained, the proposed logic-based Benders decomposition method outperforms a pure constraint programming model in terms of the number of proven optimal solutions on the tested instances under a common time limit and especially on instances with low preemption.

Keywords: Scheduling, Skills, Logic based Benders decomposition, Mixed-integer programming, Constraint programming.

1. Introduction

In the literature on the resource-constrained project scheduling problem (RCPSP), the multi-skill model introduced by [Bellenguez-Morineau and Néron \(2008\)](#) shows increasing interest as shown in a recent survey ([Snauwaert and Vanhoucke, 2023](#)). This model considers multi-skilled (MS) resources representing human resources (workers) or multi-purpose machines, and instead

Email addresses: c.juvin@tbs-education.fr (Carla Juvin), christian.artigues@laas.fr (Christian Artigues), pierre.lopez@laas.fr (Pierre Lopez)

of requiring a specific set of resources as in the RCPSP, each activity requires a set of skills and a number of resources for each skill. This allows us to model a wide range of practical applications and represent a large number of resource allocation possibilities in a compact way.

However, in the traditional multi-skill project scheduling problem (MSPSP) (Bellenguez-Morineau and Néron, 2008; Correia and Saldanha-da-Gama, 2015a; Snauwaert and Vanhoucke, 2023), simplifying assumptions consider the MS resources as standard disjunctive resources. An MS resource is assigned at any time to a single skill, and the MS resources are allocated to an activity from its start to its end. In reality, when a multi-skilled resource represents a human resource, these simplifying assumptions are too restrictive, and more flexibility is needed to accurately model the problem and obtain better solutions.

In this paper, we study a partially preemptive resource-constrained project scheduling problem with flexible multi-skilled resources. The project consists of a set of activities linked by precedence constraints. Instead of requiring specific resources, each activity has a fixed duration and requires for each skill a fixed number of MS resources that master this skill. The problem then amounts to assigning execution time periods and a set of MS resources to each activity such that the makespan of the project is minimised. Contrarily to the MSPSP, we consider that an activity may need additional cumulative resources¹ and that some activities may be totally or partially preempted. Totally preemptive activities can be classically interrupted and restarted at any time while all the required (cumulative and multi-skilled) resources are released during the interruption periods. For partially preempted activities, only the MS resources are released during activity preemption, while the cumulative resources remain unavailable for other activities. An MS resource can also be unavailable during pre-specified intervals to take into account worker breaks. These additional activity and resource constraints aim to model practical situations in complex industrial scheduling environments such as aircraft assembly line industry (Pováda et al., 2023), nuclear research facilities (Polo-Mejía et al., 2020, 2023) and train maintenance factories (Torba et al., 2024).

For example, consider the case where a cumulative resource represents an area of limited capacity where an activity requiring some heavy material is processed by a worker modelled by an MS resource. Suppose that the worker stops this activity and starts performing another activity. The material is still occupying the area that remains therefore unavailable for a third activity that would need it until the first activity is resumed and completed. This is precisely the case of a partial preemption since the MS resource is released by the partial preemption

¹The cumulative resource defined by Aggoun and Beldiceanu (1993) is widely used in Constraint Programming as a synonym of the standard renewable resource in the RCPSP literature (Hartmann and Briskorn, 2022).

while the cumulative resource remains occupied.

Due to the presence of cumulative resources such as in the resource-constrained project scheduling problem (RCPSP) and of partial preemption, the problem presented so far can be denoted as MS-RCPSP-PP.

In this paper, we consider additional levels of flexibility related to MS resources and skill requirements. The first type of flexibility, denoted AF (allocation flexibility), is linked to a relaxation of the standard MSPSP multi-skilled resource allocation constraints. These constraints state that the set of MS resources allocated to an activity remains the same from the start time to the end of the activity. Here, we assume that an MS resource can be replaced by another one at any time during the execution of an activity, provided that the skill requirements of the activity remain satisfied. This introduces a level of flexibility since the set of MS resources allocated to an activity can change over time. Such a flexibility for worker allocation also appears in the machine scheduling literature (Li et al., 2024). In the papers by Polo-Mejía et al. (2020, 2023), MS resource change was allowed only for preemptive and partially preemptive activities, while in this paper we consider this flexibility for all activity types.

The second type of flexibility, denoted SF (skill flexibility), is linked to a relaxation of the multi-skill requirement satisfaction constraints. In the standard MSPSP, the allocation of a set of resources to an activity must ensure that each resource is allocated to at most one activity at a given time and that the skill requirements of the activities are satisfied. More precisely, each resource in the allocated sets performs a single skill among the one it masters. In this paper, the constraint of assigning a single skill to a worker is relaxed considering that a worker can sometimes fulfill several skills simultaneously. This is notably the case when skills model levels of clearance in scheduling problems involving hazardous material (Polo-Mejía et al., 2023).

The problem obtained with all the above constraints is called the multi-skill resource-constrained project scheduling problem with partial preemption, allocation and skill flexibility (MS-RCPSP-PP-ASF). The goal is to propose an exact method to solve this challenging NP-hard problem.

Section 2 presents the related work and highlights our contributions. The MS-RCPSP-PP-ASF is defined in Section 3. Integer linear programming and constraint programming formulations extending the ones proposed by Polo-Mejía et al. (2020) for the MS-RCPSP-PP-SF are given. The proposed logic-based Benders decomposition (LBBD) method is described in Section 4, in which the master problem is an RCPSP relaxing the skill requirements to cumulative constraints, and the subproblem is an NP-hard skill covering problem. The logic cuts are detailed, and an algorithm to derive a feasible solution from the current master solution is given.

Computational experiments presented in Section 5 show that the LBB method is able to find a larger number of optimal solutions than the CP method on a benchmark instance set. Finally, Section 6 presents conclusions and some perspectives to be given to this work.

2. Literature Review

The only exact methods found in the literature for a closely related problem are the constraint programming (CP) and integer linear programming (ILP) formulations proposed by [Polo-Mejía et al. \(2020\)](#) for the MS-RCPSP-PP-SF, i.e. the same problem as the one considered in this paper except that no allocation flexibility was possible on non-preemptive activities. In this paper, we propose extensions of the ILP and CP formulations to the MS-RCPSP-PP-ASF that will serve as a baseline method for comparison with the proposed logic-based Benders decomposition method.

Several exact methods have been proposed for the standard MSPSP. One of the first exact methods is the branch-and-bound procedure proposed by [Bellenguez-Morineau and Néron \(2007\)](#). Given an upper bound, time windows are computed for each activity, and the branching scheme consists in splitting the time window of an activity at each node of the search tree. Once a leaf is obtained, the start time and duration of all activities are fixed. The workers are allocated to the activities by solving the NP-hard fixed interval scheduling problem obtained ([Kolen et al., 2007](#)) by mixed-integer linear programming. The paper by [Bellenguez-Morineau and Néron \(2007\)](#) also provides a polynomial algorithm to determine whether there is a feasible worker allocation to satisfy the skill requirements of a set of activities of equal duration that fully overlap. The problem can be modelled as a maximum flow problem. Several integer and mixed integer programming formulations are proposed and compared by [Almeida et al. \(2019\)](#).

Several constraint programming methods, including one using no-good learning and lazy clause generation, are proposed by [Young et al. \(2017\)](#).

Regarding mathematical decomposition methods, branch-and-price methods are considered in several papers ([Correia and Saldanha-da-Gama, 2015b](#); [Montoya et al., 2014, 2015](#)). The methods rely on time-indexed formulations of the problem and consider work patterns. A work pattern is defined as an activity start time and a set of allocated workers. Several extended formulations are proposed, considering a binary variable for each activity and each possible work pattern. Because of the exponential number of such variables, the linear programming relaxations of these formulations are solved via column generation. All these formulations yield the same subproblem for each activity and each time period aiming at finding the minimum-cost work pattern for the activity, which amounts to solving a minimum-cost maximum-flow problem.

A hybrid (logic-based) Benders decomposition method is proposed by [Yi-fan et al. \(2013\)](#).

The master problem relaxes the worker allocation problem into cumulative resources. More precisely, if W workers master a skill l , then a cumulative resource of capacity W is created and an activity requiring q workers for skill l requires q units of this resource. The usage of any worker is also modelled by a cumulative resource of capacity equal to the total number of workers. This relaxation is a standard resource-constrained project scheduling problem (RCPSP) that is solved by constraint programming. The subproblem then consists in allocating workers to fixed activities while fulfilling skill requirements, i.e. a fixed interval scheduling problem (Bellenguez-Morineau and Néron, 2007) solved by integer programming. In case of failure of the subproblem, precedence constraints are generated heuristically to remove the conflicts by analyzing the peaks in skill requirements. The method is not exact because of this heuristic cut generation. However, scheduling an RCPSP in the master problem is appealing because of the efficiency of modern CP solvers on this problem.

An exact Benders decomposition method has also been proposed for a variant of the MSPSP but with the more classical decomposition scheme where the master problem models the resource allocation problem while the subproblem models the scheduling problem. Li (2015) and Li and Womer (2009) consider a multi-skill project scheduling problem, with the objective of minimising the total worker allocation cost. Each worker has a workload limit, in terms of the total duration of her/his assigned activities, that cannot be exceeded. For each required skill, each activity requires exactly one worker that must be allocated to this activity during all its processing time. A hybrid ILP/CP Benders decomposition is proposed, where the restricted master problem is an ILP that models the minimum-cost worker/activity assignment problem with knapsack constraints to take account of workload limits. The subproblem is a feasibility CP model that searches for a feasible schedule given the worker allocation computed by the master problem. A similar decomposition is applied to a two-stage stochastic multiskilled workforce staffing and scheduling problem by Nasirian et al. (2025) and to a distributed flexible job-shop problem by Xiong and Liu (2026). The drawback of this classical scheme is that the master problem lacks information about the scheduling part.

This is why in this paper we opt for the reverse scheme proposed by Yi-fan et al. (2013) to solve the standard MSPSP. However, in the MS-RCPSP-PP-ASF, because of the flexible skill constraints allowing the checking of skill fulfilment only in each time period, the subproblem is no more a general fixed-interval scheduling problem but a particular case where a set of workers has to be allocated to a set of activities simultaneously in process during a time period. In the standard MSPSP case where a worker can perform only one skill when allocated to an activity, checking feasibility on a period basis would be solved by maximum flow problem computations in

each time period, as stated by [Bellenguez-Morineau and Néron \(2007\)](#). In the MS-RCPSP-PP-ASF, the fact that a worker can perform several skills when allocated to an activity makes the subproblem NP-hard (as demonstrated in Section 4.4.3). However, unlike [Yi-fan et al. \(2013\)](#), we define the necessary logic cuts from conflict analysis in case of subproblem infeasibility with proved convergence toward optimality.

Table 1 presents the papers on exact methods for variants of the MSPSP cited in this section according to the following classification: problem type (where WC and WL refer to worker cost and worker load constraints, as described above), the presence of (partial) preemption (column P), skill flexibility (column SF) and allocation flexibility (column AF where NP, P, PP means that allocation flexibility holds for non-preemptive, preemptive, and partially preemptive activities, respectively). The acronyms used for the exact methods are BB for dedicated branch-and-bound, BP for branch-and-price, CP/SAT for hybrid constraint programming and boolean satisfiability, LBBD for logic-based Benders decomposition. For the LBBD methods, A-MP/S-SP means that the master problem solves the resource allocation problem while the subproblem solves the scheduling problem; S-MP/A-SP means the opposite.

Reference	Problem type	P	SF	AF	Method
Bellenguez-Morineau and Néron (2007)	MSPSP	no	no	no	BB
Li and Womer (2009)	MSPSP-WC-WL	no	no	no	LBBD (A-MP/S-SP)
Almeida et al. (2019)	MSPSP	no	no	no	MILP
Montoya et al. (2014)	MSPSP	no	no	no	BP
Correia and Saldanha-da-Gama (2015b)	MSPSP	no	no	no	BP
Li (2015)	MSPSP-WC-WL	no	no	no	LBBD (A-MP/S-SP)
Young et al. (2017)	MSPSP	no	no	no	CP/SAT
Polo-Mejía et al. (2020)	MS-RCPSP-PP-SF	P, PP	yes	P,PP	CP and MILP
This paper	MS-RCPSP-PP-ASF	P, PP	yes	NP,P,PP	LBBD (S-MP/A-SP)

Table 1: Literature review on exact methods for variants of the MSPSP

3. Problem Definition

3.1. Formal Description

The multi-skill resource-constrained project scheduling problem with partial preemption, allocation and skill flexibility (MS-RCPSP-PP-ASF) considers a set of activities $\mathcal{A} = \{1, \dots, n\}$ divided into three subsets: $\mathcal{A} = \mathcal{A}^P \cup \mathcal{A}^{NP} \cup \mathcal{A}^{PP}$. Set $\mathcal{A}^P$ includes fully preemptive activities. When a fully preemptive activity is interrupted, all its allocated resources and workers are released and made available for other activities. Set $\mathcal{A}^{NP}$ consists of the non-preemptive activities. Set $\mathcal{A}^{PP}$ is the set of partially preemptive activities. When a partially preempted

activity is interrupted, a subset of its allocated resources, including all its allocated workers, is released, while a subset of its allocated resources is blocked until the activity is completed. We consider a set of time periods $\mathcal{T} = \{1, \dots, T\}$, a set $\mathcal{L} = \{1, \dots, L\}$ of skills, and a set $\mathcal{HR}$ of disjunctive resources that represent multi-skilled workers, i.e. each worker can process only one activity at a time.

For each time period $t \in \mathcal{T}$, a parameter $\pi_{w,t} \in \{0, 1\}$ indicates whether a worker $w \in \mathcal{HR}$ is available ($\pi_{w,t} = 1$) or absent ($\pi_{w,t} = 0$). A parameter $m_{w,l} \in \{0, 1\}$ indicates whether worker $w \in \mathcal{HR}$ masters skill $l \in \mathcal{L}$ ($m_{w,l} = 1$) or not ($m_{w,l} = 0$). A set $\mathcal{CR}$ of cumulative resources is also considered. Each resource $k \in \mathcal{CR}$ has a discrete availability $B_k \in \mathbb{N}$. Such resources typically represent geographical areas of limited capacities, teams of mono-skilled workers, energy power limitations, etc. Each activity $i \in \mathcal{A}$ is defined by a processing time $p_i \in \mathbb{N}$, a demand $b_{i,k} \in \mathbb{N}$ for each cumulative resource $k \in \mathcal{CR}$ and a requirement $a_{i,l} \in \mathbb{N}$ for each skill $l \in \mathcal{L}$. In addition, processing an activity i requires a minimum number of workers q_i , independently of the required skills. Moreover, for each partially preemptive activity $i \in \mathcal{A}^{PP}$ and each resource $k \in \mathcal{CR}$, a parameter $\rho_{i,k} \in \{0, 1\}$ indicates whether resource k can be released during preemption periods of activity i ($\rho_{i,k} = 0$) or not ($\rho_{i,k} = 1$). For each fully preemptive activity $i \in \mathcal{A}^P$, $\rho_{i,k} = 0$ for all $k \in \mathcal{CR}$. For each non-preemptive activity $i \in \mathcal{A}^{NP}$, $\rho_{i,k} = 1$ for all $k \in \mathcal{CR}$. Furthermore, there is a set E of precedence constraints, where $(i, j) \in E$ means that activity j cannot start before activity i is completed. Finally, our objective in MS-RCPSP-PP-ASF is to find a schedule that minimises the total duration of the project or makespan ($C_{\max}$). The MS-RCPSP-PP-ASF is NP-hard as it includes the RCPSP as a particular case.

Example 1. An example with 5 activities ($A_1, \dots, A_5$), 3 workers ($W_1, \dots, W_3$), and 1 resource (M_1) is illustrated in [Table 2](#). In this simple example, there is a precedence constraint between activities A_3 and A_5 , and an absence of worker W_1 in time period 3.

Note that this example (or part of this example) will serve as a running example throughout the paper.

For this instance, [Figure 1](#) shows a feasible solution with optimal makespan.

In this solution, the makespan is 13. Furthermore, we can see that activity A_1 is partially preempted during time period 3. This means that resource M_1 is blocked during this period, while workers W_2 and W_3 are released and allocated to activity A_3 since W_1 is not available in time period 3 ($\pi_{1,2} = 0$).

3.2. Notations

[Table 3](#) lists the sets and parameters of the MS-RCPSP-PP-ASF.

Sets	
$\mathcal{A}$	Set of activities to be scheduled ($\mathcal{A} = \mathcal{A}^P \cup \mathcal{A}^{NP} \cup \mathcal{A}^{PP}$)
$\mathcal{L}$	Set of skills $\mathcal{L} = \{1, \dots, L\}$
$\mathcal{HR}$	Set of MS resources
$\mathcal{CR}$	Set of cumulative resources
E	Set of precedence constraints
$\mathcal{T}$	Time horizon $\mathcal{T} = \{1, \dots, T\}$
Parameters	
p_i	Processing time of activity i ($p_i \in \mathbb{N}$)
B_k	Amount of resource k ($B_k \in \mathbb{N}$)
$b_{i,k}$	Amount of resource k required for executing activity i ($b_{i,k} \in \mathbb{N}$)
$a_{i,l}$	Amount of workers mastering skill l required for activity i ($a_{i,l} \in \mathbb{N}$)
$\pi_{w,t}$	Equal to 1 if worker w is available during time period t , 0 otherwise
$m_{w,l}$	Equal to 1 if worker w masters skill l , 0 otherwise
q_i	Minimum number of workers required for activity i ($q_i \in \mathbb{N}$)
$\rho_{i,k}$	Equal to 0 if k can be released during the preemption of activity i , 1 otherwise

Table 3: Sets and parameters of the MS-RCPSP-PP-ASF

$$\sum_{i \in \mathcal{A}} z_{w,i,t} \leq \pi_{w,t} \quad \forall w \in \mathcal{HR}, \forall t \in \mathcal{T} \quad (3)$$

$$\sum_{w \in \mathcal{HR}} z_{w,i,t} \cdot m_{w,l} \geq x_{i,t} \cdot a_{i,l} \quad \forall l \in \mathcal{L}, \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (4)$$

$$\sum_{w \in \mathcal{HR}} z_{w,i,t} \geq x_{i,t} \cdot q_i \quad \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (5)$$

$$\sum_{t=r_i}^{d_i} x_{i,t} \geq p_i \quad \forall i \in \mathcal{A} \quad (6)$$

$$p_i \cdot (1 - x_{j,t}) \geq \sum_{t'=t}^T x_{i,t'} \quad \forall (i, j) \in E, \forall t \in \mathcal{T} \quad (7)$$

$$x_{i,t}^- \geq x_{i,t'} \quad \forall i \in \mathcal{A}^{NP} \cup \mathcal{A}^{PP}, \forall t \in \mathcal{T}, \forall t' \leq t \quad (8)$$

$$x_{i,t}^+ \geq x_{i,t'} \quad \forall i \in \mathcal{A}^{NP} \cup \mathcal{A}^{PP}, \forall t \in \mathcal{T}, \forall t' \geq t \quad (9)$$

$$y_{i,t} = x_{i,t}^- + x_{i,t}^+ - x_{i,t} - 1 \quad \forall i \in \mathcal{A}^{PP}, \forall t \in \mathcal{T} \quad (10)$$

$$x_{i,t}^- + x_{i,t}^+ - x_{i,t} = 1 \quad \forall i \in \mathcal{A}^{NP}, \forall t \in \mathcal{T} \quad (11)$$

$$C_{\max} \geq t \cdot x_{i,t} \quad \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (12)$$

Main decision variables	
$x_{i,t}$	Equal to 1 if activity i is executed during time t , 0 otherwise
$z_{w,i,t}$	Equal to 1 if worker w is allocated to activity i during time t , 0 otherwise
$y_{i,t}$	Equal to 1 if activity i is being preempted during time period t , 0 otherwise
Auxiliary decision variables	
$x_{i,t}^-$	Equal to 1 if activity i begins in time period t or before, 0 otherwise
$x_{i,t}^+$	Equal to 1 if activity i ends during time period t or after, 0 otherwise

Table 4: Decision variables for the ILP model

The objective (1) represents the makespan minimisation. Constraints (2) ensure that, at any given time, the use of the cumulative resources does not exceed their capacity. Constraints (3) prevent workers from being allocated to an activity during their absences and to more than one activity at a time. Constraints (4) and (5) ensure that the allocation of workers to activities meets technical requirements: there are enough workers (5), and they master the necessary skills (4). Constraints (6) ensure that each activity has the correct processing time and that each activity is executed within a time window $[r_i, d_i]$ that can be computed classically by longest path length calculations in the precedence graph $(\mathcal{A}, E)$ identifying each activity by a vertex and each precedence constraint $(i, j) \in E$ by an arc with valuation p_i . Earliest start time r_i is equal to the length of the longest path among all paths ending at i in $(\mathcal{A}, E)$. Deadline d_i is equal to T minus the length of the longest path among all paths issued from i in $(\mathcal{A}, E)$. Constraints (7) express the precedence constraints between activities. If activity j is in progress at time t (i.e. $x_{j,t} = 1$), then the left-hand side of the inequality must equal zero, which implies that the right-hand side must also be zero. This directly enforces that all $x_{i,t'} = 0$ for $t' \geq t$, meaning that activity i must already be completed by time t . In other words, activity i cannot be in progress at or after the start time of activity j thus respecting the precedence relationship $(i, j) \in E$. Constraints (8) and (9) determine the value of the auxiliary variables $x_{i,t}^-$ and $x_{i,t}^+$. Constraints (10) define the preemption for partially preemptive activities, an activity $i \in \mathcal{A}^{PP}$ is preempted during time period t (i.e. $y_{i,t} = 1$), if it is not in process ($x_{i,t} = 0$) while being started ($x_{i,t}^- = 1$) and not yet completed ($x_{i,t}^+ = 1$). Constraints (11) ensure that the processing of non-preemptive activities is not interrupted. Finally, Constraints (12) define the makespan as the end of the last activity in progress.

3.4. Constraint Programming Model

To model the problem using CP, we adapt the CP model proposed by [Polo-Mejía et al. \(2020\)](#). The model uses interval decision variables to represent activities. An interval decision variable `ivar` is a high-level decision variable that encapsulates a start-time decision variable `ivar.start`, an end-time decision variable `ivar.end` and a duration decision variable `ivar.dur`. When declaring an interval variable `ivar`, domains have to be specified for `ivar.start`, `ivar.end` and `ivar.dur`, denoted $[\text{ivar.start}, \text{ivar.start}]$, $[\text{ivar.end}, \text{ivar.end}]$ and $[\text{ivar.dur}, \text{ivar.dur}]$, respectively. This concept is present in the IBM CP Optimizer, Hexaly and Google OR-Tools CP-SAT solvers. Here, we use the CP Optimizer (CPO) formalism. We first recall in Section 3.4.1 the semantics of the constraints on these interval variables that are needed to efficiently model the MS-RCPSP-PP-ASF constraints. Then we give the CP model in Section 3.4.2.

3.4.1. Constraints on interval variables

Function $\text{pulse}(\text{ivar}, h)$ is the pulse function of height h associated with an interval variable `ivar`. Consider a set I of indices such that $\{\text{ivar}_i, i \in I\}$ is a set of interval variables representing activities. Let $h_i, \forall i \in I$, an integer representing the requirement of activity i on some cumulative resource. Let H denote the capacity of the cumulative resource. The cumulative constraints stating that at each time point the total requirement of the activities in progress cannot exceed the capacity can be compactly written as:

$$\sum_{i \in I} \text{pulse}(\text{ivar}_i, h_i) \leq H$$

Note that $f = \sum_{i \in I} \text{pulse}(\text{ivar}_i, h_i)$ is a sum of pulse functions and is called a cumul function in CPO. A more restrictive cumulative constraint, $\text{alwaysIn}(f, a, b, hmin, hmax)$, given a cumul function f and a fixed discrete interval $\{a, \dots, b-1\}$, enforces $f(t) \in [hmin, hmax], \forall t \in \{a, \dots, b-1\}$. CP solvers use efficient overload check and variable domain filtering algorithms for each of the declared cumulative constraints, such as the timetabling algorithm ([Fahimi and Quimper, 2014](#)), the cumulative edge finding algorithm ([Vilím, 2009](#)) and the energetic reasoning algorithm ([Ouellet and Quimper, 2018](#)).

The $\text{endBeforeStart}(\text{ivar}, \text{jvar})$ constraints between two interval variables `ivar` and `jvar` allow us to represent the precedence constraint $\text{ivar.end} \leq \text{jvar.start}$.

Let `jvar` denote an interval variable and $\{\text{ivar}_i, i \in I\}$ a set of interval variables. The $\text{span}(\text{jvar}, \{\text{ivar}_i, i \in I\})$ constraints enforce that all intervals `ivari` are included in interval `jvar`, i.e. $\forall i \in I, \text{ivar}_i.start \geq \text{jvar.start}$ and $\text{ivar}_i.end \leq \text{jvar.end}$.

`ivar` being an interval decision variable and F being a piecewise constant function of time, constraint $\text{forbidExtent}(\text{ivar}, F)$ prevents `ivar` from overlapping any time point t such that

$$F(t) = 0$$

Finally, an interval variable can be declared as *optional* to let the solver decide whether the corresponding activity is present or absent, i.e. scheduled or not. If **jvar** is an interval variable, $\{\mathbf{ivar}_i, i \in I\}$ a set of *optional* interval variables and q an integer variable, constraint *alternative*(**jvar**, $\{\mathbf{ivar}_i, i \in I\}, q$) states that exactly q activities in set $\{\mathbf{ivar}_i, i \in I\}$ are present and fully synchronized with **jvar**: i.e. $|\{\mathbf{ivar}_i, i \in I, \mathbf{ivar}_i \text{ is present}\}| = q$ and $\forall i \in I$, if $\mathbf{ivar}_i$ is present then $\mathbf{ivar}_i.start = \mathbf{jvar.start}$, $\mathbf{ivar}_i.end = \mathbf{jvar.end}$ and $\mathbf{ivar}_i.dur = \mathbf{jvar.dur}$.

3.4.2. CP model

An interval variable $\mathbf{act}_i$ is defined for each activity $i \in \mathcal{A}$. To model preemption, as well as the possibility of changing worker allocation at any time period, activities are divided into unit-duration parts. For each activity $i \in \mathcal{A}$, p_i interval variables, denoted $\mathbf{par}_{i,v}$, $\forall v \in V_i$, where $V_i \in \{1, \dots, p_i\}$, are introduced. An optional interval variable $\mathbf{awo}_{w,i,v}$ is defined for each worker $w \in \mathcal{HR}$ and each part $v \in V_i$ of activity $i \in \mathcal{A}$, which is present in a solution only if that worker is allocated to that part of the activity. The domains of the interval variables considered are given in Table 5.

$\{\mathbf{ivar}_i, i \in I\}$	$\mathbf{ivar}_i.start$	$\mathbf{ivar}_i.start$	$\mathbf{ivar}_i.end$	$\mathbf{ivar}_i.end$	$\mathbf{ivar}_i.dur$	$\mathbf{ivar}_i.dur$
$\{\mathbf{act}_i, i \in \mathcal{A}^{NP}\}$	r_i	$d_i - p_i$	$r_i + p_i$	d_i	p_i	p_i
$\{\mathbf{act}_i, i \in \mathcal{A} \setminus \mathcal{A}^{NP}\}$	r_i	$d_i - p_i$	$r_i + p_i$	d_i	p_i	$d_i - r_i + 1$
$\{\mathbf{par}_{i,v}, i \in \mathcal{A}, v \in V_i\}$	$r_i + v - 1$	$d_i - p_i + v - 1$	$r_i + v$	$d_i - p_i + v$	1	1
$\{\mathbf{awo}_{w,i,v}, w \in \mathcal{HR}, i \in \mathcal{A}, v \in V_i\}$	$r_i + v - 1$	$d_i - p_i + v - 1$	$r_i + v$	$d_i - p_i + v$	1	1

Table 5: Interval variables parameters

The integer variable $nW_{i,v}$ indicates the number of workers allocated to the part $v \in V_i$ of activity $i \in \mathcal{A}$. Likewise, the integer variable $nS_{l,i,v}$ indicates the number of workers mastering skill $l \in \mathcal{L}$ allocated to the part $v \in V_i$ of activity $i \in \mathcal{A}$. For each skill $l \in \mathcal{L}$ a cumul function sU_l is introduced, which represents the cumulated requirement of skill l for the activities in progress at any given time. Finally, a cumul function wU represents the cumulative usage of workers by activities over time. Table 6 summarizes the notations used in the CP model, described as follows.

$$\min C_{\max} \tag{13}$$

$$C_{\max} \geq \mathbf{act}_i.end \quad \forall i \in \mathcal{A} \tag{14}$$

Decision variables	
$C_{\max}$	Integer variable related to the makespan
act_i	Interval variable between the start and the end of processing of activity i
$\text{par}_{i,v}$	Interval variable of unit duration representing the processing of the v^{th} part of activity i
$\text{awo}_{w,i,v}$	Optional interval variable of unit duration representing the processing of the v^{th} part of activity i by worker w
$nS_{l,i,v}$	Integer variable related to the number of workers mastering skill l allocated to the v^{th} part of activity i
$nW_{l,i,v}$	Integer variable related to the number of workers allocated to the v^{th} part of activity i
wU	Cumulative function representing the sum of contributions of activities in worker usage
sU_l	Cumulative function representing the sum of contributions of activities in usage of skill l
Additional Parameters	
aW_t	Number of available workers during time period t
$aS_{l,t}$	Number of available workers mastering skill l during time period t

Table 6: Constraint programming decision variables and parameters

$$\text{span}(\text{act}_i, \{\text{par}_{i,v}, \forall v \in V_i\}) \quad \forall i \in \mathcal{A} \quad (15)$$

$$\text{endBeforeStart}(\text{par}_{i,v}, \text{par}_{i,s}) \quad \forall i \in \mathcal{A}, \forall v, s \in V_i, s > v \quad (16)$$

$$\text{endBeforeStart}(\text{act}_i, \text{act}_m) \quad \forall (i, m) \in E \quad (17)$$

$$\sum_{i \in \mathcal{A}} \sum_{v \in V_i} \text{pulse}(\text{par}_{i,v}, (1 - \rho_{i,k}) \cdot b_{i,k}) + \sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, \rho_{i,k} \cdot b_{i,k}) \leq B_k \quad \forall k \in \mathcal{CR} \quad (18)$$

$$\text{noOverlap}((\text{awo}_{w,i,v}; \forall i \in \mathcal{A}, \forall v \in V_i)) \quad \forall w \in \mathcal{HR} \quad (19)$$

$$\text{forbidExtent}(\text{awo}_{w,i,v}, \pi_w) \quad \forall w \in \mathcal{HR}, \forall i \in \mathcal{A}, \forall v \in V_i \quad (20)$$

$$\text{alternative}(\text{par}_{i,v}, \{\text{awo}_{w,i,v} \mid \forall w \in \mathcal{HR}, m_{w,l} = 1\}, nS_{l,i,v}) \quad \forall i \in \mathcal{A}, \forall v \in V_i, \forall l \in \mathcal{L} \quad (21)$$

$$nS_{l,i,v} \geq a_{i,l} \quad \forall i \in \mathcal{A}, \forall v \in V_i, \forall l \in \mathcal{L} \quad (22)$$

$$\text{alternative}(\text{par}_{i,v}, \{\text{awo}_{w,i,v} \mid \forall w \in \mathcal{HR}\}, nW_{i,v}) \quad \forall i \in \mathcal{A}, \forall v \in V_i \quad (23)$$

$$nW_{i,v} \geq q_i \quad \forall i \in \mathcal{A}, \forall v \in V_i \quad (24)$$

$$wU = \sum_{i \in \mathcal{A}} \sum_{v \in V_i} \text{pulse}(\text{par}_{i,v}, q_i) \quad (25)$$

$$sU_l = \sum_{i \in \mathcal{A}} \sum_{v \in V_i} \text{pulse}(\text{par}_{i,v}, a_{i,l}) \quad \forall l \in \mathcal{L} \quad (26)$$

$$\text{alwaysIn}(wU, t_q^{aW}, t_{q+1}^{aW}, 0, aW_q) \quad \forall q \in \{1, \dots, aW\} \quad (27)$$

$$\text{alwaysIn}(sU_l, t_q^{aS}, t_{q+1}^{aS}, 0, aS_{l,q}) \quad \forall q \in \{1, \dots, |aS|\}, \forall l \in \mathcal{L} \quad (28)$$

The objective (13) is to minimise the makespan, defined by Constraints (14). Constraints (15) make sure that all unit parts of each activity are scheduled while the activity is in process. Constraints (16) state that all unit parts do not overlap each other and are ordered lexicographically to avoid trivial symmetries. Constraints (17) are the precedence constraints. The cumulative Constraints (18) guarantee that the use of cumulative resources does not exceed their capacity. Constraints (19) ensure that each MS resource cannot be allocated to more than one activity at a time. Constraints (20) make sure that each MS resource is allocated only when available. Constraints (21) and (22) guarantee that the allocation of MS resources satisfies the skills required for each activity. Similarly, Constraints (23) and (24) ensure compliance with the minimum number of workers allocated to each activity. Finally, redundant cumulative constraints are introduced to improve constraint propagation. A piecewise constant function aW that gives the total number of MS resources available during time period t can be precomputed as $aW(t) = \sum_{w \in \mathcal{HR}} \pi_{w,t}$. Similarly, consider the piecewise constant function aS_l giving the total number of MS resources available during time period t for skill l , i.e. $aS_l(t) = \sum_{w \in \mathcal{HR}, m_{w,l}=1} \pi_{w,t}$. Given a piecewise constant function f , let $|f|$ denote its number of breakpoints, and t_q^f , $q \in \{1, \dots, |f|\}$ denote the q^{th} breakpoint with $t_1^f = 0$ and $t_{|f|}^f = T$. Let f_q denote value $f(t)$ for $t \in [t_q^f, t_{q+1}^f - 1]$. Then constraints (25)–(28) are redundant cumulative constraints ensuring that the number of MS resources (respectively, the number of MS resources mastering skill l) required over time does not exceed the total available amount of MS resources (resp. the total available amount of MS resources mastering skill l) at each time t .

4. Logic-Based Benders Method

Benders decomposition (Benders, 1962) is an iterative method used to solve large optimisation problems by decomposing them into two subsets of decisions. At each iteration, a master problem (MP) considers a subset of the decision variables. The remaining variables are tackled by one or several subproblems (SP), which are simpler to solve as the master problem decision variables are already fixed. Solutions of these subproblems provide information on the relevance of the proposed master problem decisions. This information is passed to the master problem by means of additional constraints, called Benders cuts, that guide the search to promising assignments while avoiding those already tried. This process is repeated until an optimal solution is found.

In the classical Benders decomposition, Benders cuts are derived by solving the dual of the subproblem, meaning that this subproblem must be formulated as a linear program. The logic-based Benders decomposition (LBBD) introduced by Hooker (2000) generalizes this method. It

allows the subproblem to take any form, and there is no standard mechanism for generating cuts. Instead, cuts must be designed by analysing the specific structure of the problem. An extensive survey of the LBBB literature is given by [Hooker \(2019\)](#).

4.1. Problem Decomposition

The MS-RCPSP-PP-ASF involves both scheduling and assignment decisions. The structure of the problem, therefore, makes it naturally suited to decomposition. LBBB is often used in this case ([Cire et al., 2016](#)). As stated in Section 2, when using LBBB to solve such integrated scheduling and resource allocation problems, the resource allocation part is usually modelled by the master problem, while the subproblem deals with the scheduling part. The master problem is then generally solved by ILP and the subproblem by CP, in accordance with the predominance of these paradigms on the corresponding problems. In contrast, in our approach, scheduling is addressed in the master problem via CP and then, if feasible, the workers are allocated by integer linear programming in the SP.

At each iteration, the master problem provides a schedule, which defines only the time periods during which each activity is processed and minimises the makespan. This gives a lower bound (LB) for the objective function. Then, the SP consists in finding an allocation of workers to activities in each time period that satisfies both capacity and requirement constraints. If such an assignment is possible, the optimal solution is reached. Otherwise, if no assignment is possible, a heuristic is used to generate a feasible solution to the problem and thus an upper bound (UB). If this UB equals the lower bound given by the MP, the solution provided by the heuristic is optimal. Otherwise, cuts are derived and added to the master problem, which is then solved again. This process is repeated until an optimal solution is found or a stopping criterion is reached. The general LBBB scheme applied to our problem is illustrated in [Figure 2](#).

4.2. A Constraint Programming Model for the Master Problem

The master problem is the RCPSP-PP relaxation of the MS-RCPSP-PP-ASF where the allocation of workers to activities is ignored. The decision variables are therefore only the activity processing dates and the objective is to find a schedule that minimises the makespan while satisfying precedence constraints, and capacity of cumulative resources. It is modelled and solved using Constraint Programming.

It has been shown that including relaxation of the subproblem into the master problem is time efficient ([Cire et al., 2016](#)). Although the decision variables concerning the allocation of workers do not appear in this model, additional constraints are added to guide the master problem. To that purpose, skills and workers usage constraints are relaxed into cumulative resources. Note

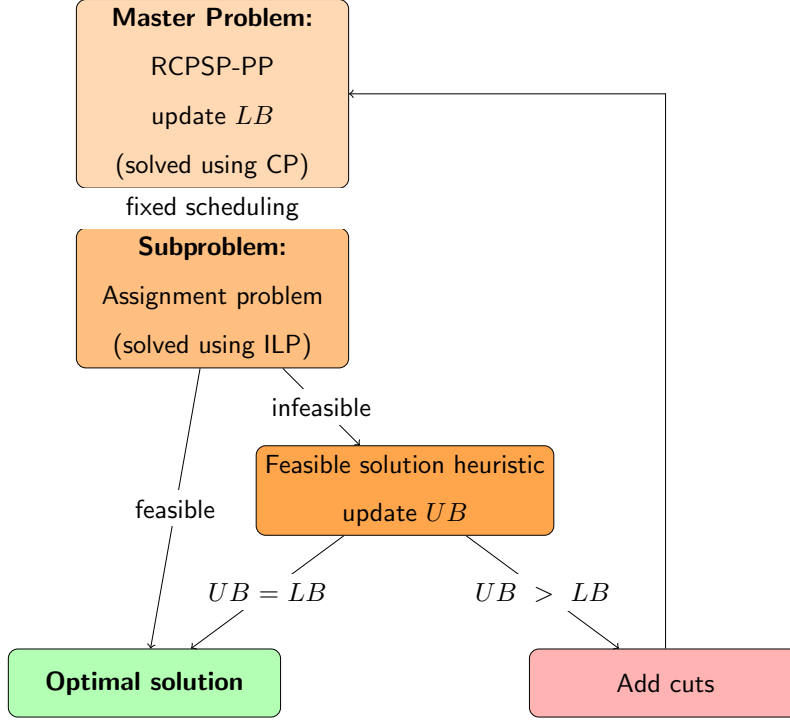


Figure 2: Decomposition scheme for the MS-RCPSP-PP-ASF

that adding such redundant cumulative resources has already been proven useful to compute lower bounds for the RCPSP (Carlier and Néron, 2007; Baptiste and Bonifas, 2018). This is the same principle as in the CP model presented in Section 3.4.2 (Constraints (25)–(28)). However, instead of considering resource capacities that vary over time due to workers’ absences (piecewise constant parameters aW and aS in Constraints (27)–(28)), we model here these absences as fixed activities. This simplifies the model, as it results in homogeneity between activities and absences, but above all it appears as helpful for designing LBBD cuts (see Section 4.5).

In the following, let $\mathcal{U}_w$ denote the set of absence periods of worker w .

Example 2. Consider again the instance of Example 1. Since there are 3 workers, we introduce a new cumulative resource called ‘workers’ with a maximum capacity of 3. We do the same for each skill. Skills 1 (c_1) and 2 (c_2) are mastered by two workers, so a new cumulative resource with a capacity of 2 is introduced for each of these two skills, as well as 2 cumulative resources with a unit capacity as other skills (c_3 to c_4) are mastered by only one worker each. We obtain 6 cumulative resources. Note that the absence of W_1 in time period $t = 3$ is modelled by fixed activity U_1 , scheduled on cumulative resources workers, c_1 and c_2 since both skills are mastered by W_1 . An example of a solution found by the master problem is described by Figure 3.

The Constraint Programming model for the Master Problem is made up of Constraints (13)–(18), which concern the scheduling part (precedence and cumulative constraints). As mentioned

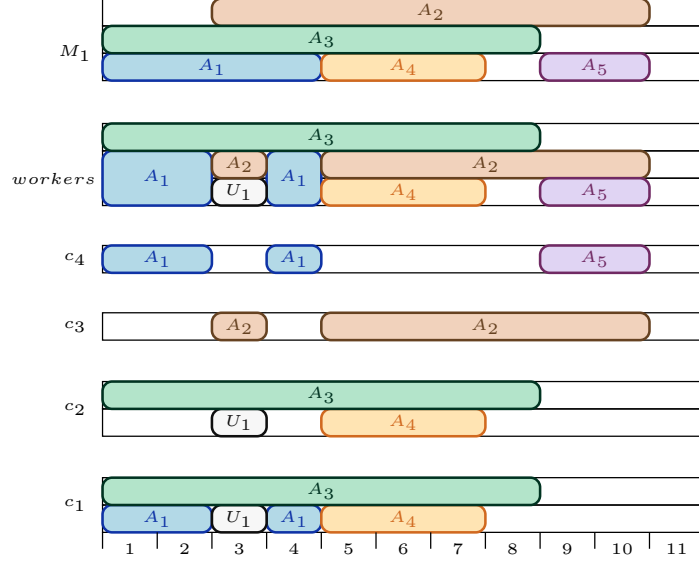


Figure 3: A master problem solution

above, new interval variables with fixed start times are introduced to take into account the absences of each worker. More precisely, $\forall w \in \mathcal{HR}, \forall u \in \mathcal{U}_w$, we introduce $\mathbf{abs}_{w,u}$ the interval variable corresponding to the u -th absence of worker w .

Taking account of these activities, skills and workers are considered as cumulative resources, which introduces a relaxation of the subproblem in the master problem with the following constraints, which ensure, for each skill, that the requirements never exceed the skill capacity:

$$\sum_{i \in \mathcal{A}} \sum_{v \in V_i} \text{pulse}(\mathbf{par}_{i,v}, a_{i,l}) + \sum_{w \in \mathcal{HR}} \sum_{u \in \mathcal{U}_w} \text{pulse}(\mathbf{abs}_{w,u}, m_{w,l}) \leq \sum_{w \in \mathcal{HR}} m_{w,l} \quad \forall l \in \mathcal{L} \quad (29)$$

This is equivalent to considering a cumulative resource per skill $l \in \mathcal{L}$ of capacity $\sum_{w \in \mathcal{HR}} m_{w,l}$ on which each activity $i \in \mathcal{A}$ requires $a_{i,l}$ units and each absence $\mathbf{abs}_{w,u}$ requires $m_{w,l}$ units. Constraint (30) ensures that the minimum worker requirement never exceeds the total number of workers, which amounts to defining a cumulative constraint of capacity $|\mathcal{HR}|$, required by each activity i with intensity q_i and each absence activity $\mathbf{abs}_{w,u}$ with intensity 1:

$$\sum_{i \in \mathcal{A}} \sum_{v \in V_i} \text{pulse}(\mathbf{par}_{i,v}, q_i) + \sum_{w \in \mathcal{HR}} \sum_{u \in \mathcal{U}_w} \text{pulse}(\mathbf{abs}_{w,u}, 1) \leq |\mathcal{HR}| \quad (30)$$

4.3. Preprocessing: Constraint Strengthening

Consider an activity i requiring a minimum of 1 worker ($q_i = 1$). If no worker masters all the skills required by an activity i , we can simply change the minimum number of workers required from $q_i = 1$ to $q_i = 2$. In the case where there is at least one worker mastering all the skills required by the activity, it may still happen that no such worker is available during

some particular periods. This means that in these periods, the effective minimum number of workers required is at least 2. Taking account of this case, we can strengthen Constraint (30) in a preprocessing phase of the master problem. Let $\mathcal{A}^{1w}$ be the set of activities that require a minimum of 1 worker and for which there are workers with all the skills but all absent during some time periods. And let $\mathcal{HR}_i$ be the set of workers who master all the skills required by $i \in \mathcal{A}^{1w}$. Constraint (30) is replaced by the following constraints:

$$\sum_{i \in \mathcal{A} \setminus \mathcal{A}^{1w}} \sum_{v \in V_i} pulse(\mathbf{par}_{i,v}, q_i) + \sum_{i \in \mathcal{A}^{1w}} \sum_{v \in V_i} pulse(\mathbf{par}_{i,v}, h_{i,v}) + \sum_{w \in \mathcal{HR}} \sum_{u \in \mathcal{U}_w} pulse(\mathbf{abs}_{w,u}, 1) \leq |\mathcal{HR}| \quad (31)$$

$$h_{i,v} + |\mathcal{HR}_i| \geq pulse(\mathbf{par}_{i,v}, 2) + \sum_{w \in \mathcal{HR}_i} \sum_{u \in \mathcal{U}_w} pulse(\mathbf{abs}_{w,u}, 1) \quad \forall i \in \mathcal{A}^{1w}, \forall v \in V_i \quad (32)$$

The contribution of a part v from an activity $i \in \mathcal{A}^{1w}$ is a decision variable $h_{i,v} \in \{1, 2\}$. If it is processed at a same time as absence of all workers from $\mathcal{HR}_i$, then the cumulative function $pulse(\mathbf{par}_{i,v}, 2) + \sum_{w \in \mathcal{HR}_i} \sum_{u \in \mathcal{U}_w} pulse(\mathbf{abs}_{w,u}, 1)$ reaches a maximum of $2 + |\mathcal{HR}_i|$ and $h_{i,v}$ is constrained to be 2 by Constraints (32).

Example 3. Worker W_1 is the only one who masters both skills c_1 and c_2 , which are required by activities A_3 and A_4 . However, during time period $t = 3$, worker W_1 is not available (with $\mathcal{U}_1 = \{1\}$). As a result, activities A_3 and A_4 require at least two workers to be performed during time period $t = 3$.

$$h_{i,v} + 1 \geq pulse(\mathbf{par}_{i,v}, 2) + pulse(\mathbf{abs}_{1,1}, 1) \quad \forall i \in \{3, 4\}, \forall v \in V_i \quad (33)$$

We could generalize this type of constraints for activities requiring a minimum number of workers greater than or equal to 2, but this requires enumerating all sets of workers that master skills of an activity, which is exponential.

4.4. Subproblems: Assignment

Once the processing dates for each activity are given by the Master Problem, workers must be allocated to each activity.

4.4.1. Time decomposition

Given that the allocation of workers can change during the processing of the activities, it suffices to make sure that there exists a feasible assignment for every time period. Thus, the subproblem can be split into a number of smaller problems. Note that if there exists a feasible assignment at a given time, this assignment remains feasible at least until another activity (including absence activities) starts. So, we can split the time horizon into periods we call delta-periods. A delta-period starts when a set of activities begins, and ends when a new set of activities begins (or when the very last activity ends).

Example 4. For instance, if we split the schedule given in Example 2 into delta-periods, we obtain the decomposition given by Figure 4.

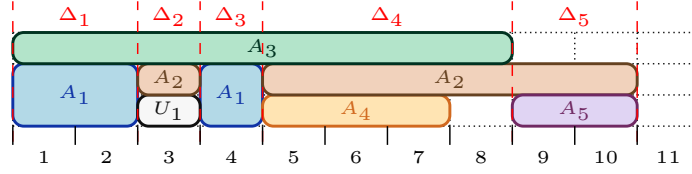


Figure 4: A delta-period decomposition from Example 2

The first delta-period, denoted Δ_1 , starts with time period $t = 1$ (start of activities A_1 and A_3). The second one, denoted Δ_2 , covers time period $t = 3$ (start of activity A_2 and absence U_1). Delta-period Δ_3 starts when activity A_1 is resumed (time period $t = 4$). Delta-period Δ_4 starts with time period $t = 5$ (start of activities A_2 and A_4). Finally, the last delta-period, denoted Δ_5 , starts with time period $t = 9$ (start of activity A_5).

Therefore, the aim of the subproblem is to check whether there exists a possible assignment for each delta-period.

4.4.2. Integer linear model

For a given delta-period Δ , we will note $\mathcal{A}_\Delta$ the set of activities starting or in progress at the beginning of Δ , and $\mathcal{HR}_\Delta$ the set of absent workers at the beginning of Δ . The aim of the subproblem is to find a feasible worker allocation for each activity. This is solved using integer linear programming. The decision variables are defined in Table 7.

Decision variables	
$y_i \in \{0, 1\}$	Equal to 1 if the assignment of activity $i \in \mathcal{A}_\Delta$ is satisfied, 0 otherwise
$z_{w,i} \in \{0, 1\}$	Equal to 1 if worker $w \in \mathcal{HR} \setminus \mathcal{HR}_\Delta$ is allocated to activity $i \in \mathcal{A}_\Delta$, 0 otherwise

Table 7: Subproblem decision variables

Then, the integer linear programming model for the assignment in the delta-period Δ is:

$$\max w_\Delta = \sum_{i \in \mathcal{A}_\Delta} y_i \quad (34)$$

$$\sum_{i \in \mathcal{A}_\Delta} z_{w,i} \leq 1 \quad \forall w \in \mathcal{HR} \setminus \mathcal{HR}_\Delta \quad (35)$$

$$\sum_{w \in \mathcal{HR} \setminus \mathcal{HR}_\Delta} z_{w,i} \geq q_i \cdot y_i \quad \forall i \in \mathcal{A}_\Delta \quad (36)$$

$$\sum_{w \in \mathcal{HR} \setminus \mathcal{HR}_\Delta} z_{w,i} \cdot m_{w,l} \geq a_{i,l} \cdot y_i \quad \forall l \in \mathcal{L}, \forall i \in \mathcal{A}_\Delta \quad (37)$$

The objective (34) is to maximise the number of assigned activities. Constraints (35) make sure that each worker cannot be allocated to more than one activity. Constraints (36) and (37) ensure that the minimum number of workers and the skill requirements of an activity are reached, respectively.

Let w_Δ^* be the optimal objective value of the subproblem in the delta-period Δ . Solving the SP allows us to know whether the set of activities $\mathcal{A}_\Delta$ can be scheduled in parallel, when workers from $\mathcal{HR}_\Delta$ are absent, or not. Indeed, if $w_\Delta^* = |\mathcal{A}_\Delta|$ then a feasible assignment is given for all activities of $\mathcal{A}_\Delta$. Otherwise, i.e. if $w_\Delta^* < |\mathcal{A}_\Delta|$, activities of $\mathcal{A}_\Delta$ cannot be all scheduled in parallel and no more than w_Δ^* of them can.

Example 5. *Based on the schedule found by the Master Problem, one can find the following assignment for the different delta-periods:*

- $\mathcal{A}_{\Delta_1} = \mathcal{A}_{\Delta_3} = \{A_1, A_3\}$: $w_{\Delta_1}^* = w_{\Delta_3}^* = 2$, W_2 and W_3 can be allocated to A_1 and W_1 to A_3 .
- $\mathcal{A}_{\Delta_2} = \{A_2, A_3\}$, $\mathcal{HR}_{\Delta_2} = \{W_1\}$: $w_{\Delta_2}^* = 1$, the assignment cannot be satisfied (W_1 being absent, A_3 requires the other two workers).
- $\mathcal{A}_{\Delta_4} = \{A_2, A_3, A_4\}$: $w_{\Delta_4}^* = 2$, the assignment cannot be satisfied (explanation is given in Example 9).
- $\mathcal{A}_{\Delta_5} = \{A_2, A_5\}$: $w_{\Delta_5}^* = 1$, the assignment cannot be satisfied (both activities require a skill mastered only by W_3).

4.4.3. Subproblem complexity

In this section, we show that the subproblem is strongly NP-hard, which justifies the use of ILP to solve it.

Theorem 1. *The subproblem is strongly NP-hard, even for $|\mathcal{A}_\Delta| = 2$.*

Proof. We consider the decision variant (*DSP*) of subproblem (34)–(37) with $w = |\mathcal{A}_\Delta|$. Consider the NP-hard SETMULTICOVER problem with a set of items to cover U , a demand d_u for each $u \in U$, and a set $\mathcal{S}$ of subsets of U . Given binary variable $x_S = 1$ if subset S is selected in the cover, the ILP model of SETMULTICOVER is:

$$\begin{aligned} \min \quad & \sum_{S \in \mathcal{S}} x_S \\ \text{s.t.} \quad & \sum_{S \in \mathcal{S}, u \in S} x_S \geq d_u & \forall u \in U \\ & x_S \in \{0, 1\} & \forall S \in \mathcal{S} \end{aligned}$$

We consider the decision variant DSETMULTICOVER where we ask for a feasible solution such that $\sum_{S \in \mathcal{S}} x_S \leq k$ with k an integer verifying $k < |\mathcal{S}|$. Note that this is equivalent to asking that the number of unselected subsets is not smaller than $|\mathcal{S}| - k$, i.e. $\sum_{S \in \mathcal{S}} (1 - x_S) \geq |\mathcal{S}| - k$.

We can build an instance of *DSP* as follows. Consider 2 activities such that $\mathcal{A}_\Delta = \{1, 2\}$, skill set $\mathcal{L} = U$, and worker set $\mathcal{HR} = \mathcal{S}$ such that $S(w)$ is the set associated with worker index w . Worker $w \in \mathcal{HR}$ masters the skills corresponding to the items that the corresponding set $S(w)$ covers, i.e. $m_{w,u} = 1$ if $u \in S(w)$. Activity 1 has a demand of $a_{1,u} = d_u$ workers for each skill $u \in U$, and no minimum number of workers ($q_1 = 0$). Activity 2 has no demand in skills ($a_{2,u} = 0, \forall u \in U$) but a minimum number of workers $q_2 = |\mathcal{S}| - k$. The *DSP* ILP can then be written:

$$\begin{aligned} z_{w,1} + z_{w,2} &\leq 1 & \forall w \in \mathcal{HR} \\ \sum_{w \in \mathcal{HR}} z_{w,1} \cdot m_{w,u} &\geq a_{1,u} & \forall u \in \mathcal{U} \\ \sum_{w \in \mathcal{HR}} z_{w,2} &\geq q_2 \\ z_{w,1}, z_{w,2} &\in \{0, 1\} & \forall w \in \mathcal{HR} \end{aligned}$$

Note that the first constraint can be replaced by equality $z_{w,1} + z_{w,2} = 1$. Indeed, workers not allocated to activity 1 can all be allocated to activity 2 since there is no upper limit for allocated workers. Hence, the *DSP* has two constraints: $\sum_{w \in \mathcal{HR}} z_{w,1} \cdot m_{w,u} \geq d_u$ for all $u \in \mathcal{U}$ and $\sum_{w \in \mathcal{HR}} (1 - z_{w,1}) \geq |\mathcal{S}| - k$. Identifying binary variable $z_{w,1}$ with the x_S variable, we obtain that the *DSP* instance has a solution if and only if the DSETMULTICOVER instance has a solution. $\square$

Note also that, as remarked in the introduction of this paper, if each worker is limited to perform only one skill for its assigned activity, as in the standard MSPSP, the *DSP* be-

comes polynomial and can be solved by a maximum flow computation in the graph defined by [Bellenguez-Morineau and Néron \(2007\)](#). The graph is tripartite with the set of activity nodes, the set of skill nodes, the set of worker nodes, plus a source and a sink node. The source node is linked to each activity node i by an arc with capacity $\sum_{l \in \mathcal{L}} a_{i,l}$. Each activity node i is linked to each skill node l such that $a_{i,l} > 0$ with an arc of capacity $a_{i,l}$. Each skill node l is linked to each worker node w such that $m_{w,l} = 1$ by an arc of capacity 1 and each worker node is linked to the sink node with an arc of capacity 1, modelling the fact that the worker can be assigned to only one skill and one activity. The DSP has a solution if and only if the maximum flow in the graph is equal to $\sum_{i \in \mathcal{A}_\Delta} \sum_{l \in \mathcal{L}} a_{i,l}$. Allowing the workers to perform more skills for their assigned activities brings more flexibility, but can greatly increase the problem difficulty.

4.5. Logic Cuts

If there exists a delta-period Δ in which no assignment is possible for all activities of $\mathcal{A}_\Delta$, a cut must be added to the master problem to avoid encountering the same conflict in subsequent iterations.

4.5.1. Activity-dependent cuts

The subproblem gives w_Δ^* as the maximum number of activities in $\mathcal{A}_\Delta$ that can be processed in parallel when all workers from $\mathcal{HR}_\Delta$ are absent. Then, when no assignment is possible, this is trivially inferred to the master problem thanks to the following constraint:

$$\sum_{i \in \mathcal{A}_\Delta} \sum_{v \in V_i} \text{pulse}(\text{par}_{i,v}, 1) + \sum_{w \in \mathcal{HR}_\Delta} \sum_{u \in \mathcal{U}_w} \text{pulse}(\text{abs}_{w,u}, 1) \leq w_\Delta^* + |\mathcal{HR}_\Delta| \quad (38)$$

Theorem 2. *Cut (38) is valid and the LBBD converges towards optimality for the MS-RCPSP-PP-ASF.*

Proof. This cut is valid in the sense that it avoids the same situation occurring in subsequent iterations and does not remove any feasible solution. More formally, the optimal solution of the master problem is infeasible for the MS-RCPSP-PP-ASF if and only if there exists a delta-period Δ such that the subproblem is infeasible. Let F denote the set of activities $\mathcal{A}_\Delta$ plus the absence activities of $\mathcal{HR}_\Delta$. The subproblem proves that no more than $w_\Delta^* + |\mathcal{HR}_\Delta| < |F|$ activities of F can be scheduled in parallel in any feasible solution to the MS-RCPSP-PP-ASF. Such a set is called a forbidden or critical set in the RCPSP literature ([Lombardi and Milano, 2012](#); [Stork and Uetz, 2005](#)). Constraint (38) precisely prevents more than $w_\Delta^* + |\mathcal{HR}_\Delta|$ activities of F being scheduled in parallel by generating a cumulative constraint, with a resource of capacity $w_\Delta^* + |\mathcal{HR}_\Delta|$ on which each activity of F requires 1 unit. Consequently, the cut is valid. Moreover, with the presence of this constraint, the same set F will not be encountered again for subsequent

iterations of the LBB method in case of another master problem infeasibility. Each added cut of type (38) removes a possible forbidden set and since there is a finite number of forbidden sets, the method converges towards optimality. $\square$

Example 6. For the infeasible delta-periods Δ_2 , Δ_4 and Δ_5 listed in Example 5, we obtain the following cuts, respectively:

$$\begin{aligned} \sum_{v \in V_2} \text{pulse}(\text{par}_{2,v}, 1) + \sum_{v \in V_3} \text{pulse}(\text{par}_{3,v}, 1) + \text{pulse}(\text{abs}_{1,1}, 1) &\leq 2 \\ \sum_{v \in V_2} \text{pulse}(\text{par}_{2,v}, 1) + \sum_{v \in V_3} \text{pulse}(\text{par}_{3,v}, 1) + \sum_{v \in V_4} \text{pulse}(\text{par}_{4,v}, 1) &\leq 2 \\ \sum_{v \in V_2} \text{pulse}(\text{par}_{2,v}, 1) + \sum_{v \in V_5} \text{pulse}(\text{par}_{5,v}, 1) &\leq 1 \end{aligned}$$

However, each cut (38) only prevents that more than $w_\Delta^* + |\mathcal{HR}_\Delta|$ activities of $\mathcal{A}_\Delta$ and the absence activities of $\mathcal{HR}_\Delta$ being scheduled in parallel while there could exist other activity subsets with similar skill requirements that cannot be scheduled in parallel. In order to strengthen their impact, more general cuts are introduced in the following section.

4.5.2. Skill-dependent cuts

One of the reasons why no assignment is possible for some delta-period is that the skills of the same workers may be required for different activities running in parallel.

Example 7. In the last delta-period, Δ_5 from the decomposition given in Example 4, activities A_2 and A_5 cannot be processed at a same time because they both need a skill (c_3 for A_2 and c_4 for A_5) that is only mastered by worker W_3 .

To address this, it is possible to define an extended skill made up of several skills. Let $S \subseteq \mathcal{L}$ be a set of skills and c_S be the associated extended skill. Activities requiring an extended skill c_S , denoted $\mathcal{A}_S$, are all activities that require at least one skill of set S . Similarly, workers who possess this extended skill, denoted $\mathcal{HR}_S$, are those who master at least one of the skills in S . As for the standard skills, the demand of all activities scheduled in parallel must not exceed the available quantity of workers possessing the extended skill.

If an infeasibility is detected in a given delta-period Δ , in which no assignment is possible for all activities of $\mathcal{A}_\Delta$, a potential subset of skills leading to this type of infeasibility can be identified by solving an ILP model. To set up this ILP model, we modify the original set of activities. Specifically, if an activity i requires multiple workers with the same skill l (i.e. $a_{i,l} > 1$), we decompose it into several sub-activities. The first sub-activity includes all skills required by the original activity with a demand of at least one, the second includes all skills

with a demand of at least two, and so on. This decomposition is applied to all activities in $\mathcal{A}_\Delta$ resulting in a new set $\mathcal{A}'_\Delta$ where each sub-activity has a skill requirement of either 0 or 1.

Example 8. Consider a skill set $\mathcal{L} = \{c_1, c_2, c_3\}$ and an activity i with skill requirements $(a_{i,l})_{l \in \mathcal{L}} = (2, 3, 1)$. The decomposition of activity i gives activities i_1, i_2 and i_3 with skill requirements $(a_{i_1,l})_{l \in \mathcal{L}} = (1, 1, 1), (a_{i_2,l})_{l \in \mathcal{L}} = (1, 1, 0)$ and $(a_{i_3,l})_{l \in \mathcal{L}} = (0, 1, 0)$.

The decision variables of the ILP are described in Table 8.

Decision Variables	
x_i	Equal to 1 if activity i has at least one selected skill, 0 otherwise
y_w	Equal to 1 if worker w has at least one selected skill, 0 otherwise
z_l	Equal to 1 if skill l is selected, 0 otherwise

Table 8: Extended skill decision variables

The model is as follows:

$$\min \sum_{l \in \mathcal{L}} z_l \quad (39)$$

$$\sum_{w \in \mathcal{HR}} y_w + 1 \leq \sum_{i \in \mathcal{A}'_\Delta} x_i \quad (40)$$

$$x_i \leq \sum_{l \in \mathcal{L}} (z_l \cdot a_{i,l}) \quad \forall i \in \mathcal{A}'_\Delta \quad (41)$$

$$y_w \geq z_l \cdot m_{w,l} \quad \forall l \in \mathcal{L}, \forall w \in \mathcal{HR} \setminus \mathcal{HR}_\Delta \quad (42)$$

The objective (39) is to determine the smallest subset of selected skills S , such that the number of activities (or sub-activities) requiring these skills is strictly greater than the number of workers with these skills (Constraint (40)). Constraints (41) define whether activities (or sub-activities) require a selected skill. Constraints (42) define whether workers master at least one selected skill.

Therefore, for a given delta-period Δ , we solve this problem and are faced with two cases:

- The problem has a solution, i.e. there exists a set of skills S for which the requirement, for activities in $\mathcal{A}_\Delta$, is higher than the available workers. We can add cut:

$$\sum_{i \in \mathcal{A}_S} \sum_{v \in V_i} pulse(\mathbf{par}_{i,v}, \max_{l \in S} (a_{i,l})) + \sum_{w \in \mathcal{HR}_S} \sum_{u \in \mathcal{U}_w} pulse(\mathbf{abs}_{w,u}, 1) \leq |\mathcal{HR}_S| \quad (43)$$

- The problem is not feasible, and we need to add an activity-dependent cut.

The interest of this cut is that some activities in $\mathcal{A}_S$ may not be present in $\mathcal{A}_\Delta$, which in this case makes the cut stronger.

Theorem 3. *Cut (43) is valid.*

Proof. Constraint (43) imposes a cumulative constraint over all parallel activities and absences involving workers from $\mathcal{HR}_S$. For each activity $i \in \mathcal{A}_S$, at least $\max_{l \in S}(a_{i,l})$ workers from $\mathcal{HR}_S$ are required to cover the skill demands associated with the extended skill c_S . Indeed, a worker can cover several different skills for the same activity and $a_{i,l}$ defines the number of workers mastering skill l required for activity i . $\square$

Example 9. In Example 7 for delta period Δ_5 , $S = \{c_3, c_4\}$, $\mathcal{A}_S = \{A_1, A_2, A_5\}$ and $\mathcal{HR}_S = \{W_3\}$, and Cut (43) results in:

$$\sum_{v \in V_1} \text{pulse}(\mathbf{par}_{1,v}, 1) + \sum_{v \in V_2} \text{pulse}(\mathbf{par}_{2,v}, 1) + \sum_{v \in V_5} \text{pulse}(\mathbf{par}_{5,v}, 1) \leq 1$$

where $\mathbf{par}_{1,v}, v \in V_1$, $\mathbf{par}_{2,v}, v \in V_2$ and $\mathbf{par}_{5,v}, v \in V_5$ are respectively the interval variables of unit duration representing the processing of the parts of activities A_1 , A_2 and A_5 . This cut is obviously stronger than the cut of type (38) since activity A_1 contributes to the cumulative resource of capacity 1 while it is not present in the cut of type (38) given in Example 6 for delta-period Δ_5 .

In delta-period Δ_4 from the decomposition given in Example 4, no assignment is possible. Indeed, activities A_3 and A_4 require both skills c_1 and c_2 , and only the worker W_1 has these two skills. So, when this worker is allocated to one of these two activities, the other needs at least two workers to meet all its requirements, and there is no worker available for A_2 . However, there is no combination of skills that can be used to write a cut of type (43) for this situation. Therefore, it is necessary to add the cut of type (38), which is given for Δ_4 in Example 6.

4.6. Feasible Solution Determination Heuristic

The main weakness of this method is that it does not provide intermediate feasible solutions; the only feasible solution given by an SP is optimal. Consequently, before reaching optimality, no upper bound is known. To overcome this and obtain a feasible solution at each iteration, a CP-based local search algorithm and a schedule generation scheme (SGS) are proposed.

4.6.1. CP-based Heuristic

At each iteration, the MP gives the processing dates of each activity and the SP gives a worker allocation for a maximum number of activities. The CP-based heuristic relies on the extended

CP model (13)–(28), with fixed processing dates and worker allocation for all activities except those that the SP could not assign, along with their successors.

Note that, in order to limit the number of parts of activities of unit duration that are not pre-assigned, the objective function of the SP (34) is slightly modified. A lexicographic objective function is used, which first maximises the number of assigned activities and then the sum of the tails of the assigned activities, the tail of an activity being defined as the sum of the durations of its successors.

4.6.2. SGS Heuristic

The SGS heuristic algorithm performs serial scheduling by processing activities in order of their start time in the MP and iteratively assigning them to the earliest feasible time intervals as proposed by Polo-Mejía et al. (2023). Model (39)–(42) is used to check whether a worker assignment is possible.

5. Computational Experiments

The aim of this section is to evaluate the impact of the decomposition method on problem solving and to compare the two exact methods on different problem instances.

5.1. Instances

First tests are based on instances proposed by Polo Mejía (2019). They consist of four sets depending on the proportion of activities associated with a given type of preemption (non-preemptive, partially preemptive, preemptive). Table 9 details the activity distributions for the different types of instances (Set A: highly preemptive; Set B: highly partially preemptive; Set C: highly non-preemptive; Set D: equally distributed). Each set comprises 50 instances. The dataset therefore consists of 200 instances in total. These are generated randomly with the following parameters: all instances have 50 activities ($|\mathcal{A}| = 50$) lasting between 5 and 15 time units ($p_i \in [5, 15]$), 10 skills ($|\mathcal{L}| = 10$), between 5 and 8 cumulative resources ($|\mathcal{CR}| \in [5, 8]$), and 8 workers (multi-skilled resources) ($|\mathcal{HR}| = 8$) divided into two teams. Although the original instances contain release dates and due dates, we do not take them into account in this study. In addition, the density of precedence relationships is low.

We also tested our methods on randomly generated non-preemptive instances derived from the PSPLIB library (Sprecher and Kolisch, 1996). In this library, we selected 40 instances named j301_X, j601_X, j901_X, and j1201_X with $X \in \{1, \dots, 10\}$ (the most resource- and network-constrained configuration in the j30/j60/j90/j120 series), comprising 30, 60, 90, and

Preemption type	Set A	Set B	Set C	Set D
Non-preemptive	10 %	10 %	80 %	33.3 %
Partially preemptive	10 %	80 %	10 %	33.3 %
Preemptive	80 %	10 %	10 %	33.3 %

Table 9: Distribution of preemption types for instances of the MS-RCPSP-PP-ASF

120 activities, respectively, which have on average: a network complexity (NC), that is a number of arcs per node, of 1.5; a resource factor (RF) of cumulative resources needed by an activity of 0.25; and a resource strength (RS) equal to 0.2, which means that the available resources are very limited. The generation parameters involve a number of skills taken in $[10, 15, 20]$ and a number of workers in $[8, 12, 20]$. Globally, $40 \times 3 \times 3 = 360$ instances are generated. Each worker masters each skill with a probability of 65 % and each activity i needs each skill l with a probability of 20 % (18 % chance that $a_{i,l} = 1$ and 2 % chance that $a_{i,l} = 2$). All other parameters concerning precedence constraints, activity durations, and cumulative resource requirements of the original instances are kept.

5.2. Experimental Results

The algorithms presented are implemented in Python 3.11, using the solvers CPLEX and CP Optimizer (CPO) 22.12 for MILP models and CP models, respectively. Each experiment resolution is executed by a single thread, on an Intel Xeon E5-2695 v4 processor running at 2.1 GHz, with a CPU time limit of 1 hour.

5.2.1. Comparison of LBBD variants

First, we compare the performance of different LBBD variants. In order to evaluate the relevance of the preprocessing presented in Section 4.3 and the skill-dependent cuts proposed in Section 4.5.2, we tested three variants described in Table 10. The first variant (V1) is the LBBD presented in Section 4 without including preprocessing or skill-dependent cuts, while the second variant (V2) includes preprocessing, and the third variant (V3) additionally includes skill-dependent cuts. V1, V2, and V3 use the CP-based heuristic, while V4 is similar to V3, but the CP-based heuristic is replaced by the SGS heuristic.

Table 11 shows the number of instances solved to optimality and the mean CPU time to obtain them, the mean internal gap, and the mean gap to the best known lower bound over all instances, for each variant of the LBBD method. The internal gap, for instance ι and method μ is defined as: $(UB_{\iota,\mu} - LB_{\iota,\mu})/UB_{\iota,\mu}$, and the gap to the best LB, for instance ι and method μ is defined as: $(UB_{\iota,\mu} - LB_{\iota,best})/UB_{\iota,\mu}$, where $LB_{\iota,best} = \max_{\mu \in \{V1, V2, V3, V4, CP\}}(LB_{\iota,\mu})$.

	LBBD variant			
	V1	V2	V3	V4
Preprocessing	–	✓	✓	✓
Activity-dependent Cuts	✓	✓	✓	✓
Skill-dependent Cuts	–	–	✓	✓
CP-based Heuristic	✓	✓	✓	–
SGS Heuristic	–	–	–	✓

Table 10: Description of LBBD variants

	Number of optima	Mean solving time (s)	Mean gap (%)	Mean gap to best LB (%)
V1	40	528	4.68	4.68
V2	58	579	4.87	4.71
V3	70	749	4.17	4.02
V4	70	863	3.68	3.53

Table 11: Results for the four variants of the LBBD method (time limit: 1 hour)

Table 11 highlights the progressive impact of the proposed LBBD enhancements. The baseline variant (V1), which involves neither preprocessing nor skill-dependent cuts, solves only 40 instances to optimality and has the largest average gaps. Note that for V1, the internal gap and the gap to the best lower bound are identical. This indicates that, for this variant, the lower bound obtained is always the best among all methods. Adding preprocessing to V2 significantly improves performance, increasing the number of optimal solutions to 58 and reducing the internal and external gaps. Introducing skill-dependent cuts in variant V3 further strengthens the method, enabling 70 instances to be solved optimally and also yielding smaller average gaps. Variant V4, which replaces the CP-based heuristic with the SGS heuristic while retaining preprocessing and skill-dependent cuts, achieves the best overall results. It solves as many instances to optimality as V3 (70) with a slight increase in computational time, but attains the lowest average gaps.

5.2.2. Focus on the best LBBD variant

In the following, we consider only the most efficient variant of the LBBD method (V4, with preprocessing, skill-dependent cuts, and SGS heuristic). Table 12 shows the number of instances solved to optimality, the mean gap over all instances, the mean CPU time for instances solved

to optimality, the mean number of iterations, and the mean number of cuts added to the MP, for variant V4.

	LBBD V4				
	Number of optima	Mean gap (%)	Mean solving time (s)	Mean number of iterations	Mean number of cuts
Set A	11	3.64	822	6.7	71
Set B	14	4.18	988	6.5	65
Set C	30	2.96	807	13.3	76
Set D	15	3.93	724	8.3	72
Total	70	3.68	863	8.7	71

Table 12: Results for the LBBD variant V4 (with preprocessing, skill-dependent cuts and SGS heuristic, time limit: 1 hour)

The solving rate is practically the same for each group of instances, but for highly non-preemptive instances (set C) the number of instances solved to optimality is actually the highest. In addition, instances from set C have a smaller average gap. We also note that the number of iterations is the greatest for set C. This can be explained by the fact that the MP is easier for instances from set C, as most activities do not need to be split into unit parts, making it easier for the MP to produce an optimal schedule. Since most instances are not solved to optimality, less time spent per iteration means more iterations within the allotted time.

Figure 5 shows the evolution of the number of proven optimal solutions over a two-hour period. Although a large number instances are solved to optimality within the first few minutes, this number continues to increase over time, demonstrating the potential of the method.

5.2.3. Comparison with CP

To test the effectiveness of the LBBD method in solving the problem under study, we now compare it with another exact solution method, in this case Constraint Programming (CP). Note that the results obtained with the MILP model presented in Section 3.3 are not reported due to poor performance. This method provides no feasible solution for 27% of the instances, and obtains no optimal solution or a better bound (UB or LB) than LBBD and CP for any of the instances.

Table 13 shows the number of optima found by LBBD V4 and CP, for each set of instances, and compares the lower bound (LB) and the upper bound (UB) obtained by the two methods.

CP solves 18 out of 200 instances, being much less efficient than the decomposition method,

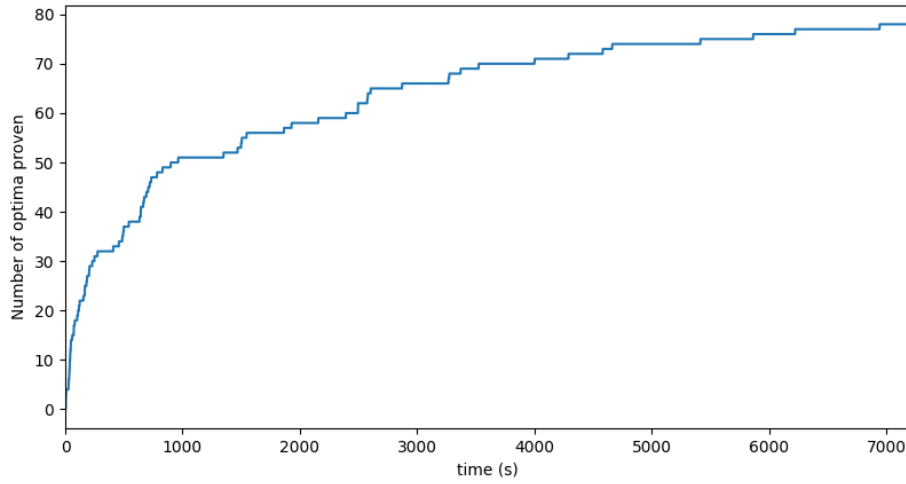


Figure 5: Number of optima proven by LBBD V4 over time

	LBBD V4			CP		
	Number of optima	Number of best UBs	Number of best LBs	Number of optima	Number of best UBs	Number of best LBs
Set A	11	9	15	10	31	5
Set B	14	13	14	2	32	4
Set C	30	31	16	1	17	0
Set D	15	17	16	5	28	1
Total	70	70	61	18	108	10

Table 13: Comparison of the LBBD variant V4 and CP (time limit: 1 hour)

for which the number of instances solved to optimality is nearly 4 times larger. CP obtains a better LB for only 10 of the 200 instances, compared with 61 for LBBD (for the remaining 129 instances, both methods obtained the same value). This shows that the main strength of the LBBD method is finding good LBs, which is the weak point of CP. However, the results for UB are more balanced. For sets A, B and D, the highly preemptive, highly partially preemptive and balanced instances respectively, CP gets more than twice as many best UBs. This can be explained by the fact that CP is often effective at finding high-quality feasible solutions. On the other hand, the weaker UBs produced by the LBBD method can be attributed to the limited number of iterations performed on these instances. With fewer opportunities to identify a feasible solution, the chances of identifying high-quality UBs are reduced. For highly non-preemptive instances (set C), LBBD also dominates CP for this indicator, obtaining a better

UB for more than 60 % of instances.

Figure 6 provides additional comparison elements. On the 14 instances on which both CP and LBBd proved optimality, LBBd is much faster. On the other hand, the average gap from the best LB on all instances is only slightly better for LBBd. This is due to a dramatically poor performance of CP on a few instances, whereas, as already mentioned, it obtains a good UB on the other instances.

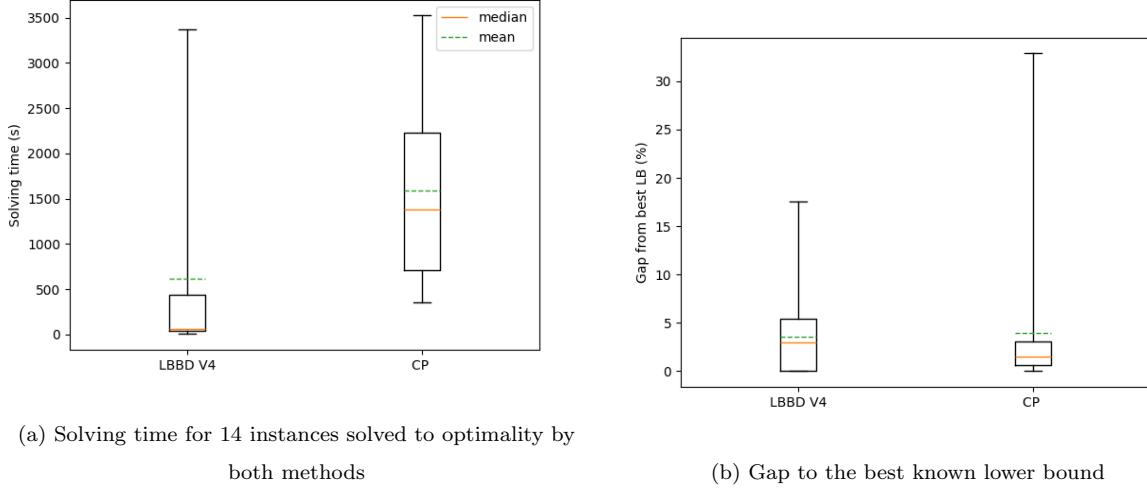


Figure 6: Additional comparison elements between the LBBd variant V4 and CP

In order to evaluate the scalability of the LBBd method and have further insights on the comparison with CP, Figure 7 presents the results of LBBd V4 and CP on the modified PSPLIB instances. The figure shows the number of proved optimal solutions in function of the number of activities, the number of skills, and the number of workers. We can see that, for both methods, in terms of the number of proven optimal solutions, the instance difficulty increases with the number of activities and skills, while it decreases with the number of workers. The results also reveal that LBBd outperforms CP in all configurations. CP is unable to prove the optimality of instances of 90 or more activities.

5.2.4. Limited number of preemptions

In practice, allowing an unlimited number of preemptions can lead to schedules that are difficult to implement. To address this possible issue, we can define an upper limit $maxPre_i$ on the number of preemptions allowed for each activity i . In our CP models (MP, CP-based heuristic, pure CP model), for each part $v \in \{1, \dots, p_i - 1\}$ of activity i , we introduce a boolean variable $pre_{i,v}$, equal to 1 if activity i is preempted after part v , 0 otherwise, along with the following constraints:

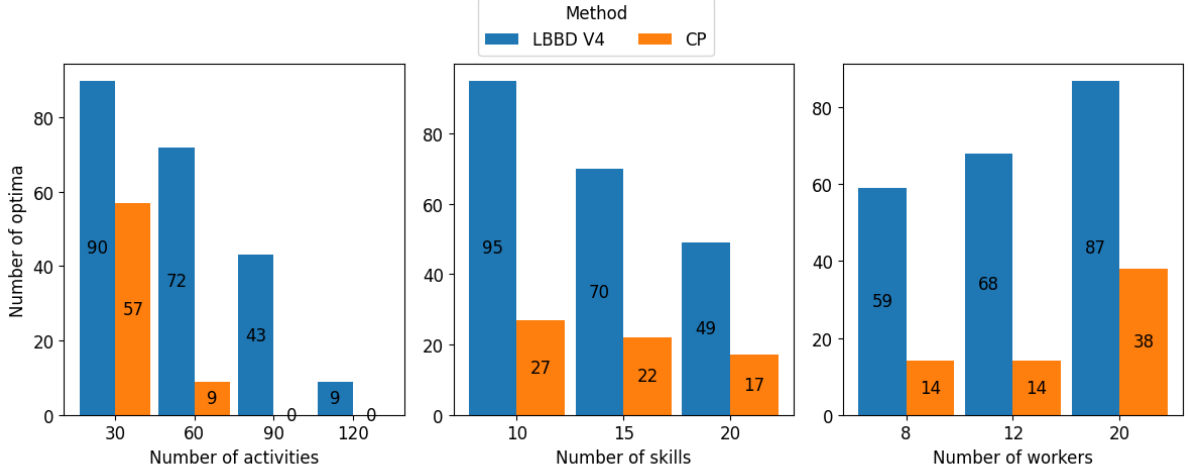


Figure 7: Number of proved optimal solutions for LBBD V4 and CP according to instance sizes

$$\neg pre_{i,v} \Rightarrow StartOf(par_{i,v+1}) \leq EndOf(par_{i,v}) \quad \forall i \in \mathcal{A}, v \in \{1, \dots, p_i - 1\} \quad (44)$$

$$\sum_{v \in V_i} pre_{i,v} \leq maxPre_i \quad \forall i \in \mathcal{A} \quad (45)$$

$$pre_{i,v} \in \{0, 1\} \quad \forall i \in \mathcal{A}, v \in \{1, \dots, p_i - 1\} \quad (46)$$

Table 14 shows the number of optima found by CP and LBBD V3² when the number of preemptions is limited to 2 per activity, for each set of instances, and compare the lower bound (LB) and the upper bound (UB) obtained by the two methods.

	LBBD V3			CP		
	Number of optima	Number of best UBs	Number of best LBs	Number of optima	Number of best UBs	Number of best LBs
Set A	9	18	12	0	24	8
Set B	11	23	12	1	25	8
Set C	26	38	16	0	11	0
Set D	14	25	16	3	20	3
Total	60	104	56	4	80	19

Table 14: Comparison of the LBBD variant V3 and CP with at most 2 preemptions per activity

Limiting the number of preemptions makes the problem more difficult for both methods: the

²CP-based heuristic is used here because the SGS variant with a limited number of preemptions is not implemented.

number of optima proved is lower (see Table 13). However, LBBD clearly remains more efficient than CP on this criterion. The difference between CP and LBBD on the LB quality decreases, although LBBD remains largely dominant on set C and also clearly dominant on the other sets. CP remains better than LBBD in terms of best UBs for sets A and B. LBBD now dominates CP on both sets C and D, and has the highest overall number of best UBs. LBBD therefore appears to be more robust to the introduction of the limit on the number of preemptions.

6. Conclusion

This paper deals with a variant of the multi-skill project scheduling problem, called MS-RCPSP-PP-ASF, which incorporates partial preemption, and flexibility over resource allocation and skill requirements. We propose an exact logic-based Benders decomposition approach that uses constraint programming to solve a scheduling master problem and integer linear programming to solve assignment subproblems. Preprocessing and logic-based cuts are proposed to enhance the solving process. Results of computational experiments on 560 benchmark instances show that our propositions are efficient, outperform a pure CP model in terms of proved optimal solutions, and perform particularly well on instances with low levels of preemption.

Although the proposed method achieves good results, it still has difficulties in obtaining good upper bounds for highly preemptive instances. Consequently, future research could incorporate more efficient heuristics to find feasible solutions. Finally, it would be interesting to study the proposed decomposition method for other variants of the multi-skill scheduling problem, especially for the standard case, where the allocation must remain the same throughout the processing of the activities and each worker can cover only one skill.

References

- Aggoun, A. and Beldiceanu, N. (1993). Extending CHIP in order to solve complex scheduling and placement problems. *Mathematical and Computer Modelling*, 17(7):57–73. [https://doi.org/10.1016/0895-7177\(93\)90068-A](https://doi.org/10.1016/0895-7177(93)90068-A)
- Almeida, B. F., Correia, I., and Saldanha-da-Gama, F. (2019). Modeling frameworks for the multi-skill resource-constrained project scheduling problem: A theoretical and empirical comparison. *International Transactions in Operational Research*, 26(3):946–967. <https://doi.org/10.1111/itor.12568>
- Baptiste, P. and Bonifas, N. (2018). Redundant cumulative constraints to compute preemptive bounds. *Discrete Applied Mathematics*, 234:168–177. <https://doi.org/10.1016/j.dam.2017.05.001>

- Bellenguez-Morineau, O. and Néron, E. (2007). A branch-and-bound method for solving multi-skill project scheduling problem. *RAIRO-Operations Research*, 41(2):155–170. <https://doi.org/10.1051/ro:2007015>
- Bellenguez-Morineau, O. and Néron, E. (2008). Multi-mode and multi-skill project scheduling problem. In Artigues, C., Demassey, S., and Néron, E., editors, *Resource-Constrained Project Scheduling: Models, Algorithms, Extensions and Applications*, pages 149–160. Wiley Online Library. <https://doi.org/10.1002/9780470611227.ch9>
- Benders, J. (1962). Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4(1):238–252. <https://doi.org/10.1007/BF01386316>
- Carlier, J. and Néron, E. (2007). Computing redundant resources for the resource constrained project scheduling problem. *European Journal of Operational Research*, 176(3):1452–1463. <https://doi.org/10.1016/j.ejor.2005.09.034>
- Cire, A. A., Coban, E., and Hooker, J. N. (2016). Logic-based benders decomposition for planning and scheduling: A computational analysis. *The Knowledge Engineering Review*, 31(5):440–451. <https://doi.org/10.1017/S0269888916000254>
- Correia, I. and Saldanha-da-Gama, F. (2015a). A modeling framework for project staffing and scheduling problems. In Schwindt, C. and Zimmermann, J., editors, *Handbook on Project Management and Scheduling Vol. 1*, pages 547–564. Springer. https://doi.org/10.1007/978-3-319-05443-8_25
- Correia, I. and Saldanha-da-Gama, F. (2015b). A note on “branch-and-price approach for the multi-skill project scheduling problem”. *Optimization Letters*, 9:1255–1258. <https://doi.org/10.1007/s11590-014-0804-0>
- Fahimi, H. and Quimper, C.-G. (2014). Linear-time filtering algorithms for the disjunctive constraint. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 28(1). <https://doi.org/10.1007/s10601-018-9282-9>
- Hartmann, S. and Briskorn, D. (2022). An updated survey of variants and extensions of the resource-constrained project scheduling problem. *European Journal of Operational Research*, 297(1):1–14. <https://doi.org/10.1016/j.ejor.2021.05.004>
- Hooker, J. N. (2000). *Logic-based methods for optimization: Combining optimization and constraint satisfaction*. John Wiley and Sons, New York. <https://doi.org/10.1002/9781118033036>

- Hooker, J. N. (2019). Logic-based benders decomposition for large-scale optimization. *Large scale optimization in supply chains and smart manufacturing: Theory and applications*, pages 1–26. https://doi.org/10.1007/978-3-030-22788-3_1
- Kolen, A. W., Lenstra, J. K., Papadimitriou, C. H., and Spieksma, F. C. (2007). Interval scheduling: A survey. *Naval Research Logistics*, 54(5):530–543. <https://doi.org/10.1002/nav.20231>
- Li, H. (2015). Benders decomposition approach for project scheduling with multi-purpose resources. In Schwindt, C. and Zimmermann, J., editors, *Handbook on Project Management and Scheduling Vol. 1*, pages 587–601. Springer. https://doi.org/10.1007/978-3-319-05443-8_27
- Li, H., Peng, J., and Wang, X. (2024). An efficient two-stage optimization algorithm for a flexible job shop scheduling problem with worker shift arrangement. *Computers & Operations Research*, 171:106785. <https://doi.org/10.1016/j.cor.2024.106785>
- Li, H. and Womer, K. (2009). Scheduling projects with multi-skilled personnel by a hybrid MILP/CP Benders decomposition algorithm. *Journal of Scheduling*, 12(3):281–298. <https://doi.org/10.1007/s10951-008-0079-3>
- Lombardi, M. and Milano, M. (2012). A min-flow algorithm for minimal critical set detection in resource constrained project scheduling. *Artificial Intelligence*, 182:58–67. <https://doi.org/10.1016/j.artint.2011.12.001>
- Montoya, C., Bellenguez-Morineau, O., Pinson, E., and Rivreau, D. (2014). Branch-and-price approach for the multi-skill project scheduling problem. *Optimization Letters*, 8:1721–1734. <https://doi.org/10.1007/s11590-013-0692-8>
- Montoya, C., Bellenguez-Morineau, O., Pinson, E., and Rivreau, D. (2015). Integrated column generation and lagrangian relaxation approach for the multi-skill project scheduling problem. In Schwindt, C. and Zimmermann, J., editors, *Handbook on Project Management and Scheduling Vol. 1*, pages 565–586. Springer. https://doi.org/10.1007/978-3-319-05443-8_26
- Nasirian, A., Zhang, L., Costa, A. M., and Abbasi, B. (2025). Multiskilled workforce staffing and scheduling: A logic-based Benders’ decomposition approach. *European Journal of Operational Research*, 323(1):20–33. <https://doi.org/10.1016/j.ejor.2024.11.033>
- Ouellet, Y. and Quimper, C.-G. (2018). A $O(n \log^2 n)$ checker and $O(n^2 \log n)$ filtering algorithm for the energetic reasoning. In *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*, pages 477–494. Springer. https://doi.org/10.1007/978-3-319-93031-2_34

- Polo Mejía, O. (2019). *Operational research approach for optimising the operations of a nuclear research laboratory*. PhD thesis, Institut National des Sciences Appliquées de Toulouse.
- Polo-Mejía, O., Artigues, C., Lopez, P., and Basini, V. (2020). Mixed-integer/linear and constraint programming approaches for activity scheduling in a nuclear research facility. *International Journal of Production Research*, 58(23):7149–7166. <https://doi.org/10.1080/00207543.2019.1693654>
- Polo-Mejía, O., Artigues, C., Lopez, P., Mönch, L., and Basini, V. (2023). Heuristic and metaheuristic methods for the multi-skill project scheduling problem with partial preemption. *International Transactions in Operational Research*, 30(2):858–891. <https://doi.org/10.1111/itor.13063>
- Pováda, G., Alvarez, N., and Artigues, C. (2023). Partially preemptive multi skill/mode resource-constrained project scheduling with generalized precedence relations and calendars. In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.CP.2023.31>
- Snauwaert, J. and Vanhoucke, M. (2023). A classification and new benchmark instances for the multi-skilled resource-constrained project scheduling problem. *European Journal of Operational Research*, 307(1):1–19. <https://doi.org/10.1016/j.ejor.2022.05.049>
- Sprecher, A. and Kolisch, R. (1996). Psplib-a project scheduling problem library. *European journal of operational research*, 96(1):205–216. [https://doi.org/10.1016/S0377-2217\(96\)00170-1](https://doi.org/10.1016/S0377-2217(96)00170-1)
- Stork, F. and Uetz, M. (2005). On the generation of circuits and minimal forbidden sets. *Mathematical programming*, 102:185–203. <https://doi.org/10.1007/s10107-004-0512-0>
- Torba, R., Dauzère-Pérès, S., Yugma, C., Gallais, C., and Pouzet, J. (2024). Solving a real-life multi-skill resource-constrained multi-project scheduling problem. *Annals of Operations Research*, pages 1–46. <https://doi.org/10.1007/s10479-023-05784-7>
- Vilím, P. (2009). Edge finding filtering algorithm for discrete cumulative resources in $\mathcal{O}(kn \log n)$. In *International Conference on Principles and Practice of Constraint Programming (CP)*, pages 802–816. Springer. https://doi.org/10.1007/978-3-642-04244-7_62
- Xiong, F. and Liu, H. (2026). Logic-based Benders decomposition methods for the distributed flexible job shop scheduling problem. *European Journal of Operational Research*, 329(3):778–797. <https://doi.org/10.1016/j.ejor.2025.08.039>

- Yi-fan, W., Fu-quan, S., Shi-xin, L., and Di, C. (2013). A new method to solve project scheduling problems with multi-skilled workforce constraints. In *2013 25th Chinese Control and Decision Conference (CCDC)*, pages 2408–2412. IEEE. <https://doi.org/10.1109/CCDC.2013.6561342>
- Young, K. D., Feydy, T., and Schutt, A. (2017). Constraint programming applied to the multi-skill project scheduling problem. In *Principles and Practice of Constraint Programming: 23rd International Conference (CP 2017), Melbourne, VIC, Australia, August 28–September 1, 2017, Proceedings 23*, pages 308–317. Springer. https://doi.org/10.1007/978-3-319-66158-2_20